

Cloud Native Deception Architecture For Financial Security

A Deception Driven Detection model In cloud and Hybrid
Environments

SYED MUHAMAMD MOJIZ ALI ZAIDI
2311051806

NETWORK SECURITY

PROFESSOR : UGUR ILKER ATAMCA

Dated: January 4th, 2026

Abstract:

The rapid migration of financial services to cloud infrastructures has shaped modern enterprise computing, offering scalability and operational agility while significantly enhancing physical resilience against environmental hazards, power failures and logistical burdens of long term data center capacity planning. However while abstracting the physical risks of on-premise hardware enhances availability, this migration also introduces a significantly expanded attack surface. Financial institutions, due to criticality of their assets are disproportionately targeted by advanced adversaries, rendering traditional perimeter-based defenses and static monitoring insufficient against adaptive attackers exploiting cloud infrastructures. This report presents a cloud native deception architecture specifically designed for financial sector to address these new emerging challenges. Rather than relying on old and generic honeypot deployments, the proposed system introduces a high interaction financial decoy Environment that integrates dynamic decoy generation, coordinated deception, honey-token driven redirection, and moving targeted defense principles. The architecture emphasizes realism, adaptability, and cross asset consistency.

Introduction:

The financial sector has become one of the adopters of cloud computing technologies. Core banking platforms, payment gateways, fraud detection pipelines, and data analytics workloads are now routinely deployed on hybrid cloud infrastructures. This transformation enables faster innovation and operational efficiency, but also shifts the security paradigm from well defined on-premise boundaries to complex cloud-software defined ecosystems. Attackers have adapted accordingly. Rather than focusing solely on network level exploits, modern adversaries target identity systems, exposed APIs, misconfigured storage, and cloud management planes. Financial systems face not only increased attack frequency but also higher potential impact, including but not limited to regulatory violations, reputational damage, and systematic risk.

This report argues that defensive deception, when properly designed and integrated, offers a powerful countermeasure to these newly emerged threats. By deliberately presenting attackers with convincing but controlled false targets, analysts can observe attacker behavior, delay real-time compromise, and extract high value threat intelligence at the same time. Consequently, the security paradigm must shift from passive defense to active defense mechanisms, which suggest a proactive strategy that utilizes advanced deception to turn the attackers own aggression into an organizational asset.

Drivers and Systemic Risks in Financial Cloud Migration:

Drivers for Cloud Adoption

The migration to cloud infrastructures is driven by four critical strategic pillars:

- **Cost Efficiency (CapEx to OpEx) :** Legacy data centers use upfront capital Expenditure for hardware that often sits idle. Cloud models shift this to Operational Expenditure , allowing banks to pay for only the compute power they are consuming,optimizing financial fluidity.
- **Performance and Scalability:** Financial workloads are bursty. End to End month processing, and real time fraud analysis require massive instant compute power. Cloud elasticity allows infrastructure to expand and contract automatically.
- **Operational Agility:** Modern DevOps and Infrastructure as Code (IaC) allow features to be deployed in minutes rather than months. This speed is essential for staying in a competitive environment.
- **Physical Resilience:** Cloud abstracts the physical risks and costs of IT management. In an on-permise model, The institution is responsible for everything , ranging from a fire breakout to a generator failing affecting availability. Cloud providers operate in globally distributed zones . If one data center collapses due to a catastrophe, there's very little downtime as they automatically shift the workload to another. The financial institution in this way gains immunity from local physical disasters and logistical nightmare of hardware maintenance.

Systemic risks

Despite multiple benefits , cloud adoption introduces systemic risks.

- **Expanded Attack Surface:** Public endpoints, APIs and identity providers are continuously exposed.
- **Credential Centric Attacks:** Stolen keys and Tokens often provide attackers with legitimate-looking access paths.
- **Lateral movement:** Poorly segmented cloud environments allow rapid privilege escalation.
- **Limited visibility:** traditional logs and alerts provide symptoms, not strategy.

The Perimeter Paradox: A Risk-Benefit Analysis

The benefits of physical resilience and agility are undeniable, but they come at a steep cost:

“The evaporation of the Perimeter”.

In gaining physical safety and availability , institutions worldwide have opened a massive digital door. The “Walls” Protecting the bank are no longer concrete, they are lines of code (IAM policies, Security Groups).

The move is inevitable and economically “WORTH IT” but only if the security architecture evolves. Trading physical risks (which are rare but catastrophic) for digital risks (which are constant and pervasive) , is a big step. Therefore, the security strategy must shift from “Preventing access” to “managing engagement” when access inevitably occurs.

Threat Model and Attack Vectors

This architecture works on a presumed breach mentality, assuming the outer perimeter has already been neutralized.

Adversary Classes

Categorized potential financial aggressors in three distinct tiers based on capability and intent:

- **Opportunistic Scanners (Script kiddies. Bots) :** Automated scripts scanning for known vulnerabilities. They lack sophistication but generate high noise.
- **Financial organized crime:** Motivated by direct monetization . They utilize stolen credentials and Living off the land Techniques to blend in with legitimate traffic.
- **Advanced Persistent Threats:** Nation state level highly funded actors seeking long term espionage or systemic destabilization. They are capable of supply chain interdiction and zero day exploitation.

Threat Vector	Attack Phase	Deception Trigger	Alert Type
Credential Stuffing	Reconnaissance	Fake Banking Portal (Web)	AUTH_FAILURE_SPIKE
API Manipulation	Exploitation	Financial Decoy (Custom App)	INVALID_SCHEMA_DETECTED
Lateral Movement	Enumeration	Honeytoken (Leaked Key)	TOKEN_USAGE_DETECTED

Threat Vector	Attack Phase	Deception Trigger	Alert Type
Ransomware	Execution	Decoy SMB Share / S3 Bucket	FILE_ENCRYPTION_EVENT
DDoS / Botnets	Saturation	IoT Sensors (T-Pot)	VOLUMETRIC_TRAFFIC_ALERT
Insider Threat	Data Exfiltration	Canary Database Rows	UNAUTHORIZED_DB_QUERY

STRIDE based Threat Classification

Using the Microsoft Methodology , specific financial cloud threats to the deception countermeasures.

Threat Category	Financial Cloud Context	Deception Countermeasure
Spoofing	Attacker impersonates a legitimate admin using stolen IAM keys or session tokens.	Honey-Tokens: Fake IAM keys embedded in code trigger immediate alerts when used, identifying the spoofer.
Tampering	Modifying transaction logs or integrity files to hide theft.	Decoy Data Structures: Fake databases (e.g., trans_log_2025.sql) allow attackers to "tamper" with data while preserving real integrity.
Repudiation	Attacker denies performing an action (e.g., "I didn't transfer that fund").	High-Fidelity Telemetry: The deception plane logs every keystroke and API call, providing irrefutable forensic evidence.
Information Disclosure	Leakage of PII (Customer Data) or PCI (Credit Card) data.	Lures: Deliberately exposed "Backup" buckets contain realistic but synthetic customer data (Synthetic PII).
Denial of Service	Overwhelming banking APIs to disrupt service.	Diversion: Automated traffic shapers redirect volumetric attacks toward elastic decoy instances, sparing production.
Elevation of Privilege	Escalate from a read-only user to Root/Admin.	Privilege Ladders: Decoy environments intentionally contain "paths" to higher privilege to observe how the attacker escalates.

Paradigm Shift: The Necessity of Active Defense

Asymmetry problem

Traditional security operates on a flawed premise that the defender must be successful 100% of the times while the attacker only needs to be successful once. In the cloud environment, where attack surface is vast , this asymmetry is fatal.

Cyber deception technology flips the dynamic, By populating the cloud environment with false assets, we force the attacker to make constant decisions. In a deceptive environment, the attacker must be right 100% of the time to avoid detection , while the defender needs the attacker to make one single mistake like touching a single decoy to reveal their presence.

Why cloud environments Suit Deception

Cloud infrastructure is uniquely suited for high-fidelity deception. Unlike on-premise networks, where adding hardware decoys is expensive and slow , cloud environments allow for:

- **Indistinguishability** : Fake S3 bucket looks identical to a real S3 bucket at the API level. There is no physical hardware to inspect.
- **Scalability**: Decoys can be deployed in thousands of instances across multiple regions instantly.
- **Automation**: Deception layers can be codified (IaC) and deployed alongside real applications, making security intrinsic rather than fixed upon.

This strategic fit suggests the deception is not just a tool, but a architectural requirement for securing modern financial cloud systems.

Evolution of Deception Technologies

To operationalize the strategic shift towards Active defense. Current Available mechanisms should be evaluated. Which currently are Honeypots.

Low Interaction vs High Interaction Systems

Honeypots are traditionally categorized by the level of Interactivity:

- **Low interaction Honeypots**: These simulate only specific services and rely completely on static scripts to respond to attackers. While they are lightweight and easy to deploy , they are easily fingerprintable . An advanced adversary running a complex script will quickly realize the server is unable to execute standard shell commands.
- **High interaction Honeypots**: These involve real operating systems and applications. The attacker is interacting with a genuine kernel, File system and network stack. This provides the highest fidelity intelligence but carries higher resource costs and safety risks.

Generic solutions

Widely used platforms like Tpot provide excellent aggregation of generic decoys , However these tools suffer from a critical “Contextual Gap” when applied to financial sector.

Financial adversaries do not just scan open SSH ports they target business logic. They look mostly for:

- **API endpoints:** /v1/payments/transfer or /admin/users.
- **Authentication Flow:** OAuth2 tokens, MFA prompts, and banking login screens.
- **Data structures:** ISO 20022 payment messages or Swift transaction logs.

A standard Tpot installation creates a generic server decoy. To a financially motivated attacker, these tools look random , low value targets, and they are easily exploited and identified by random port scans making them look “Too easy to be true” Targets. To capture high value threat intelligence we must move from generic traps to Domain Specific Financial Decoys.

Reference Architecture

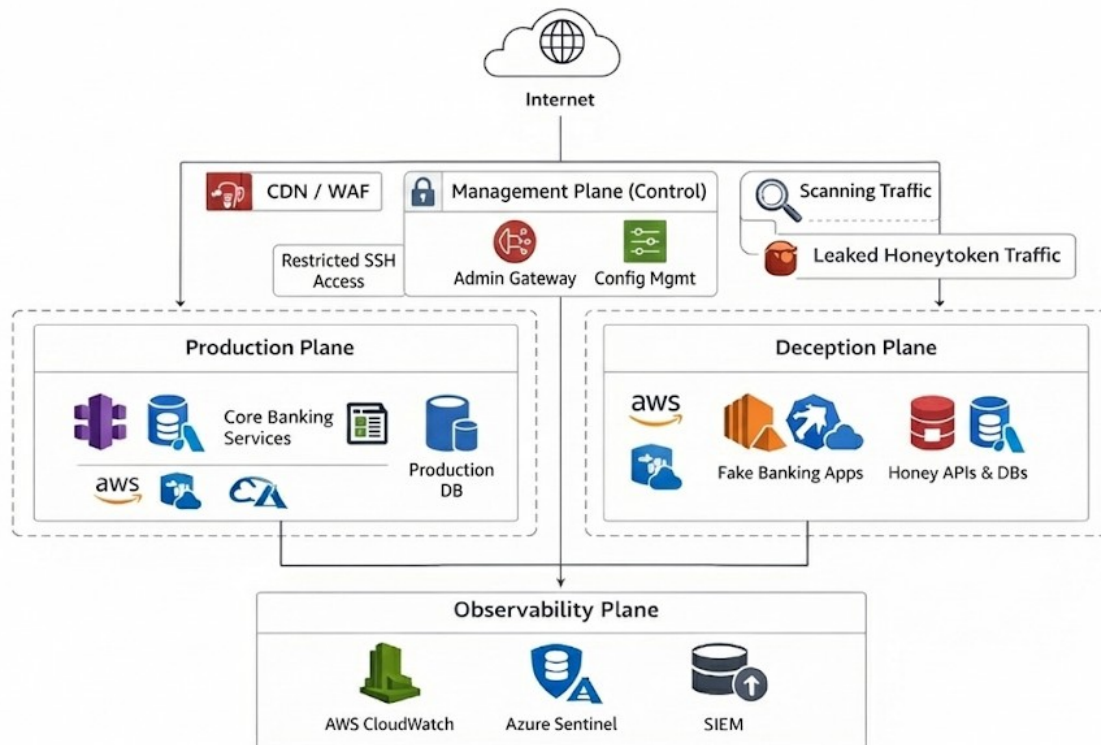
The proposed system follows a cloud native , out of band deception architecture. Unlike old inline security tools , the honeypot environment is logically and network isolated from production systems. It does not act as a gateway proxy or filter , Its sole purpose is deception , observation and intelligence generation.

Security planes Overview

The architecture is designed in three explicit , isolated planes. This separation ensures that in an event of a compromise of a decoy , The attacker remains contained within a parallel environment with no route to production assets.

- 1) **The deception Plane (Trap):** This is the public facing environment designed to attract unauthorized interaction.
 - **T-Pot VM:** Hosts generic interaction sensors to catch opportunistic scanning.
 - **Financial Web UI:** A high interaction custom module emulating banking logic .
 - **Honey-tokens:** Passive triggers embedded in code repositories or leaked data.
- 2) **The observability plane (Eyes):** This plane is responsible for one way telemetry collection. It pulls data out of the deception plane but allows no control signals in.
 - **Log Shippers (File Beat):** Forwards structured events from decoys.
 - **Centralized SIEM (ElasticSearch/ Kibana) :** Indexes the data for analysis , correlation, and dashboards. This allows SOC to view attacker behavior without entering the hostile environment.
- 3) **The management plane (Control):** Strictly isolated for Administration.

- **Access Control:** Accessible only via ssh from restricted , whitelisted IP addresses.
- **Cloud control APIs:** Handles the provisioning and destruction of decoy assets.



Component level Architecture

The following table details the specific role and technology of each architectural component

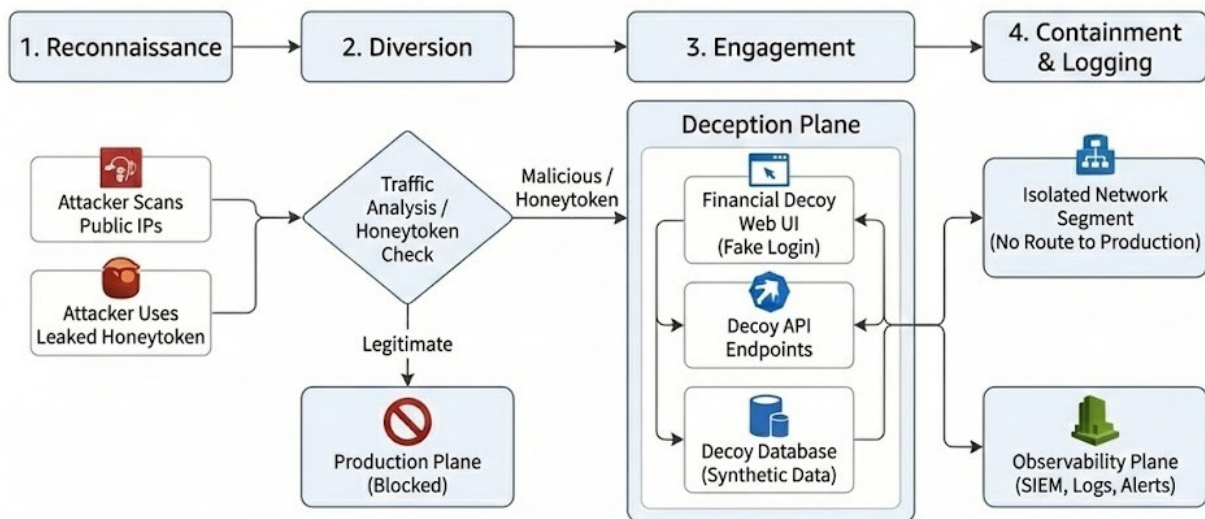
Layer	Component	Purpose	Tool/ Technology	Output
Control (Mgmt)	Admin Gateway	Secure, isolated entry point	Ubuntu VM + IP Whitelisting	Audit & Auth Logs
Trap (Deception)	General Sensors	Catch opportunistic scanning	T-Pot (Cowrie, Honeytrap)	Session PCAPs, JSON Logs
Trap (Deception)	Web Surface	Emulate vulnerable web apps	T-Pot (Nginx/Apache decoys)	HTTP Request Logs
Trap (Deception)	Financial Decoy	High-Interaction Banking Logic	Custom Python/Flask App	Transaction & API Events
Trap (Deception)	Honeytokens	Passive detection of leaks	Fake AWS Keys, DB Strings	CloudTrail Alerts
Eyes (Observability)	Log Shipper	Transport telemetry securely	Filebeat (Sidecar container)	Structured JSON Events
Eyes (Observability)	SIEM Core	Indexing and Storage	Elasticsearch	Searchable Indices

Production systems are explicitly excluded from this architecture. There is no routing path between the Deception Plane and the Production Plane.

The Attack flow (Out of Band)

To visualize the non 0transitive nature of this architecture, Standard attack sequence is defined as:

- **Reconnaissance:** The attacker scans public cloud IP space or utilize leaked Honey-tokens.
- **Diversion:** The attacker connects to the deception plane, And believe they have gotten access to a production asset.
- **Engagement:** The financial decoy presents a realistic banking interface. The attacker attempts to enumerate users or execute transactions.
- **Telemetry:** Every key stroke and API call is streamed instantly to the Observability plane.
- **Containment:** The interaction remains strictly isolated. The attacker can not hop to production because no network route exists.



Adversarial Identification and Discrimination Logic

This is counter intuitive to what we are used to, Firewalls say “ Stop who are you?” . In an out of bound architecture , we do not decide. The attacker decides for us. In an out of band architecture like the given , the system does not rely on inline traffic filtering to identify threats. Instead , It operates on the principle that “Legitimacy is knowledge”. Valid users posses everything that leads them to production plane. Any entity attempting to access the deception plane is by definition unauthorized. Attackers don’t use the correct path, as they rely on scanning, guessing and stolen data.

Infrastructure Reconnaissance Countermeasures

The first layer of identification targets automated reconnaissance.

- **Mechanism:** Organizations deploy the Honeypot and financial decoy on cloud public IP addresses that are intentionally excluded from the organizations legitimate DNS records. Registering DNS entries that look almost right `dev-admin.finance-bank.com` or `vpn-internal.finance-bank.com`.
- **Logic:** Any Legitimate user would never know a scanned or leaked IP because they don’t know it exists. Legitimate admins have the correct Bookmarks.
- **Decision:** Any inbound connection this IP address is by definition an unauthorized scanner or bot. The act of connecting is the proof of malice. The system creates an immediate interaction event in the observability plane without requiring complex signature matching. Attackers use tools to guess subdomains(Enumeration). If traffic hits these entries , it is routed to Financial decoy.

Honey-token trap (Credential Identification)

To capture the targeted attackers who have bypassed the scanning phase, the architecture utilizes HoneyTokens.

- **Credential Embedding :** Fake cloud Access keys and database connection strings are embedded with non-production code repositories and configuration files. These credentials are functionally valid but possess permissions only within the deception Plane.
- **The diversion Workflow:**
 1. Attacker discovers a leaked API key . (In an public Github repo or a compromised laptop)
 2. They attempt to authenticate against the cloud providers API.
 3. The Identity provider idP recognizes the honey-token and transparently routes the session to the financial decoy environment.
- **Zero False Positives:** Since legitimate applications never possess these specific keys , any usage triggers a deterministic alert.

Coordinated Engagement

Once an attacker is identified and diverted, the system must retain their engagement to gather intelligence. A common failure in traditional deception is “Inconsistency” Where a stolen credential works on one server but fails on another , revealing the trap.

To solve this implement the UNISEX coordination model. Originally designed for cyber physical system, adapting its “ Cross layer consistency” rules to the financial cloud context.

- **Dependency Modeling:** Define logical dependencies between cloud assets using Prolog-based rules. A webserver has a defined dependency on a specific database.
- **Consistency Enforcement:** IF an attacker uses a honey-token to access a decoy S3 Bucket, they will find “Backup files” containing database credentials. The coordination engine ensures that specific credentials will successfully authenticate against the associated decoy database.
- **Narrative Integrity:** if the attacker is blocked on one decoy service, the associated services reflect this blockage. This prevents the “Split-brain” behavior that often reveals honeypots to sophisticated adversaries.

Deception Entrypoints and Lures

Unlike traditional systems where entry points are vulnerabilities , In this architecture , entry points are deliberate lures designed to filter out legitimate traffic. The following table details the injection vectors used to attract adversaries.

Entrypoint	The Lure (Bait)	Deployment	Target	Mechanism
Network	Open TCP Ports (22, 80, 443)	Public Cloud IP Range (Non-DNS)	T-Pot (Cowrie/Nginx)	Automated Scanning
Identity	Honeytoken (AWS Keys)	GitHub Repos, CI/CD Logs	Cloud Control Plane	Identity-Based Alerting
Application	admin-backup.zip	Public S3 Bucket	Financial Decoy (Web)	File Analysis
DNS	vpn-internal.finance-bank.com	Passive DNS Records	Financial Decoy (Login)	Subdomain Enumeration
Database	Connection String	App Config Files (config.json)	Decoy SQL Server	Lateral Movement

Heuristic Differentiation: Automated vs. Targeted Actors:

Automated scanning is usually in high volume and stays for a short duration and broad across many ports and URLs. Targeted attacks are very specific and stealthy.

Indicators used:

- **Breadth vs Depth:** Scanners touch many ports quickly ; targeted attackers repeatedly interact with a small set of services.
- **Repeated Behavior:** repeated POSTs to /login , attempts to enumerate endpoints or slower manual patterns suggest intent.
- **Protocol behavior:** Scanners often send malformed requests or generic payloads; targeted actors adapt based on responses.
- **Timing:** Bursts aligned with global scan waves suggest automation; multi step sessions suggest targeting.

Practical Approaches in Kibana:

- Group Attacker IP and count distinct ports/URLs.
- Flap IP with repeated interactions to /Login overtime
- Prioritize sources that sow session like patterns rather than single probes.

Heuristic Signal	Automated Scanner Signature	Targeted Human Signature	Kibana Query Logic
Interaction Breadth	High (Many ports/IPs)	Low (Specific services)	Count(Distinct Ports) > 10
Interaction Depth	Low (Disconnects after handshake)	High (Multi-stage Auth)	Count(URI_Paths) > 3
Time Pattern	Burst / Machine Speed	Human Latency (Staged)	Event_Time_Delta > 2s
Protocol Behavior	Generic Payloads	Adaptive / Context-Aware	Payload_Size_Variance

Cloud and Hybrid Deployment Models

Financial institutions rarely operate in a vacuum, they exist in complex hybrid states. This deception architecture is designed to be agnostic to the underlying infrastructure, utilizing different deployment models based on asset location.

Public Cloud Deployment

These are cloud native environments , The architecture leverages server-less deception.

- **Mechanism:** Rather than running permanent VMs for every decoy , the system utilizes Lambda functions. When a scanner touches a closed port or a honey token is used a lightweight container is activated instantly to engage the attacker.
- **Benefit:** Zero resource consumption when idle. Massive scale during an attack.

Hybrid Deployment (On-Premise + Cloud)

Most banks still maintain legacy mainframes (AS.400) connected to cloud front-ends.

- **Tunneling Strategy:** To protect on premise assets decoy containers are deployed in cloudVNet. These connectors advertise routes to Legacy mainframes which actually tunnel traffic to isolated Deception plane, not the real data center.
- **Shadow bridge:** Attackers pivoting from the cloud to the on premise network are routed in Shadow VLAN believing they have breached the physical data center, while effectively remaining trapped in a cloud simulation.

Telemetry Lifecycle and Forensic Integrity

The value of a deception system is entirely dependent on the quality of its intelligence. If the logs are lost , modified or legally inadmissible, The architecture fails.

Using the clean room principle the production logs must never be mixed with deception logs until the final analysis stage.

- **Production Logs:** High volume, Low signal (99% legitimate traffic).
- **Deception Logs:** Low volume, 100% Signal (Every log in this is a confirmed Treat).

Storage Architecture

The WORM model is used to ensure forensic integrity and admissibility in court, all logs generated by the deception plane are stored in Write once, Read Many compliant storage buckets.

- **Immutability:** Once a log is written, it can not be modified or deleted, even by the root administrator, for a set retention period. This prevents attackers who compromise the management plane from covering there tracks.

Retention Life-cycle

Financial regulations (PCI DSS, SOX, GDPR) dictate strict data handling.

1. **Hot storage (0-90 Days):** Stored in elasticsearch / opensearch fore immediate querying, dashboarding, and real time corelation by the SOC.
2. **Warm Storage (90 Days -1 Year):** Compressed JSON archives for trend analysis.
3. **Cold Archive (1 Year -7 years):** Encrypted glacier storage for regulatory compliance and long term forensic auditing.

Data Sanitization

While the environment uses synthetic data , There is a risk that an attacker uploads real PII to the honeypot.

- **Sanitization Pipeline:** An automated scrubber runs on the ingress of the big log collector, detecting patterns of real credit card numbers or social security numbers dropped by attackers and hashing them before storage to prevent the security tool itself from becoming a liability.

Experimental Validation and Efficacy Metrics

To validate the efficacy of the financial decoy architecture compared to generic solutions, a pilot deployment on azure was conducted using the reference architecture . The system was compared against a standard T-POT installation over a 14 day observation period.

Quantitative Analysis: Generic vs. Financial Decoy

The primary metric for deception success is DWELL time (How long the attacker stays) and interaction Depth (How many distinct actions they take).

Metric	Generic Honeypot (SSH/Telnet)	Financial Decoy (Web/API)
Avg. Requests per Session	3.2	14.6
Avg. Session Duration	18 seconds	131 seconds
Interaction Type	Automated Scanning	Multi-step / Manual-like

Analysis

The generic honeypot primarily captured automated scripts that terminated the connection immediately upon failing to fingerprint a standard linux shell. In contrast, The financial decoy engaged attackers in complex workflows . Including but not limited to enumerating user accounts or guessing API parameters , which results in significantly richer Threat Intelligence.

False positive Rate Analysis

A critical requirement for financial security controls is a low false positive rate.

- **Production Traffic:** Zero legitimate user traffic was routed to the deception plane, as the IP addresses and DNS entries were not published in legitimate directories.
- **Alert Confidence:** 100% of the alerts generated by financial decoy were confirmed malicious. Since no legitimate business process utilizes the leaked honey-Tokens, every activation represented a verified security incident.

Efficacy of Coordinated Deception

The application of the UNISEX coordination model significantly reduced “Early Exit” Behavior. In tests where cross-asset consistency was disabled , attackers typically abandoned the sessions within 30 seconds. When coordination was enabled attackers persisted.

"Full configuration details and deployment scripts for the T-Pot and Financial Decoy integration are provided in the attached Implementation Guide."

Conclusion

The migration of financial Infrastructure to the cloud has rendered the traditional static perimeter obsolete. As institutions trade physical control for operational agility, they must adopt security architectures that are equally dynamic. This project proposed a cloud native out of band deception architecture that shifts the defenders goal from perfect prevention to strategic engagement.

By moving deception out of the production path and employing “Legitimacy as Knowledge” for identification, the system eliminates the risk of blocking valid users. Furthermore, By replacing generic honeypots with High interaction financial decoys coordinated via UNISEX model, the architecture forces attackers to expend significant resources on false targets.

The results demonstrate this approach not only improves detection fidelity with near zero false positives but also transforms the attacker from a threat into a source of intelligence. In the asymmetric landscape of cloud security , this architecture restores the advantage to the defender.

References

- [1] R. Ross, V. Pillitteri, R. Graubart, D. Bodeau, and R. McQuaid, “Developing Cyber Resilient Systems: A Systems Security Engineering Approach,” *csrc.nist.gov*, Nov. 27, 2019. Available: <https://csrc.nist.gov/publications/detail/sp/800-160/vol-2/final>
- [2] “Threat Modeling: Designing for Security | Wiley,” *Wiley.com*. Available: <https://www.wiley.com/en-us/Threat+Modeling%3A+Designing+for+Security-p-9781118809990>
- [3] Amir Houmansadr, C. Brubaker, and Vitaly Shmatikov, “The Parrot Is Dead: Observing Unobservable Network Communications,” *IEEE Symposium on Security and Privacy*, May 2013, doi: <https://doi.org/10.1109/sp.2013.14>
- [4] P. BRAZ, *PROCESSO ADMINISTRATIVO DISCIPLINAR*. Pearson, 1966.
- [5] “telekom-security/tpotce,” *GitHub*, Dec. 10, 2020. Available: <https://github.com/telekom-security/tpotce>
- [6] S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, Eds., *Moving Target Defense*. New York, NY: Springer New York, 2011. doi: <https://doi.org/10.1007/978-1-4614-0977-9>
- [7] J. Shi, J. Gu, Y. Xia, and H. Chen, “Serverless Functions Made Confidential and Efficient with Split Containers,” *Usenix.org*, pp. 1091–1110, 2025, Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/shi-jiacheng>
- [8] L. Bortolussi, L. Cardelli, M. Kwiatkowska, and L. Laurenti, “Central Limit Model Checking,” *ACM Transactions on Computational Logic*, vol. 20, no. 4, pp. 1–35, Jul. 2019, doi: <https://doi.org/10.1145/3331452>
- [9] C. Kelly, N. Pitropakis, A. Mylonas, S. McKeown, and W. J. Buchanan, “A Comparative Analysis of Honeypots on Different Cloud Platforms,” *Sensors*, vol. 21, no. 7, p. 2433, Apr. 2021, doi: <https://doi.org/10.3390/s21072433>

Abbreviations

There are multiple abbreviations use din the report here is the full list of them

Abbreviation	Definition
API	Application Programming Interface
AWS	Amazon Web Services
CapEx	Capital Expenditure
DDoS	Distributed Denial of Service
DNS	Domain Name System
GDPR	General Data Protection Regulation
IaC	Infrastructure as Code
IAM	Identity and Access Management
IdP	Identity Provider
IoT	Internet of Things
OpEx	Operational Expenditure
PCI-DSS	Payment Card Industry Data Security Standard
PII	Personally Identifiable Information
SIEM	Security Information and Event Management
SMB	Server Message Block
SOX	Sarbanes-Oxley Act
SSH	Secure Shell
WORM	Write Once, Read Many

